

GeoQuery — Technical Report

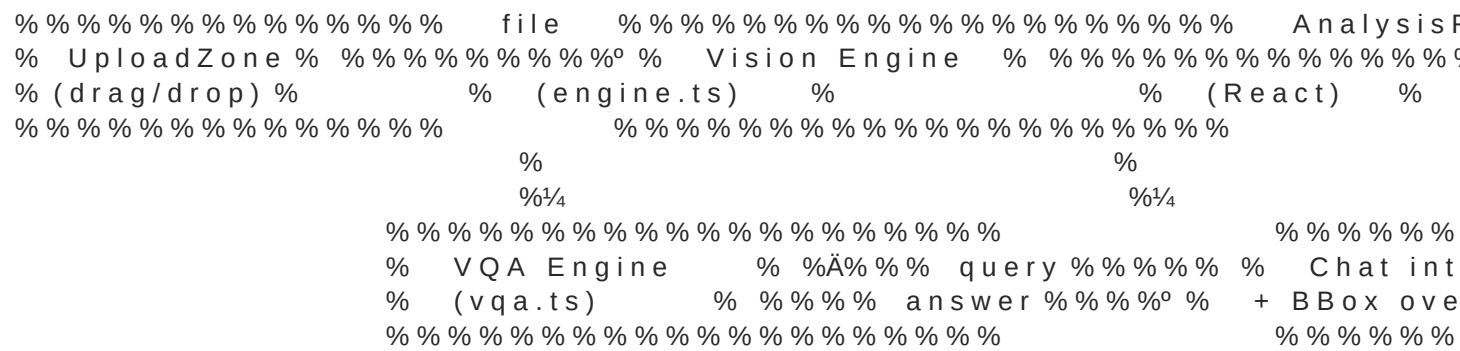
Natural Language Understanding of RGB Imagery
Mock Inter IIT Tech Meet 15.0 · IIT Ropar Internal Preparatory Edition
Inspired by ISRO SAC, Inter IIT Tech Meet 14.0

1. Abstract

GeoQuery is a deployable, fully client-side web application that lets a non-expert user upload a standard RGB photograph and interrogate it through a single conversational interface. It delivers the three capabilities mandated by the problem statement — automatic image captioning, object detection with bounding boxes, and visual question answering (VQA) over six target land-cover classes — with no paid commercial vision API in the inference path. The entire pipeline runs in the browser on the HTML Canvas 2D API, making the system trivially deployable as static assets and fully privacy-preserving (no image ever leaves the user's device).

2. System Design

2.1 Architecture Overview



The application is a React 18 + Tailwind CSS v4 single-page app. There is no server component; all computation happens in the client. The three functional modules are cleanly separated:

Module	File	Responsibility
Vision engine	src/app/lib/engine.ts	Image decode/resize, segmentation, bounding boxes, caption
VQA engine	src/app/lib/vqa.ts	Intent inference, grounded answer generation, multi-turn context
Colour utils	src/app/lib/color.ts	RGB! HSV, human colour naming
Report export	src/app/lib/exportReport.ts	Annotated-image + transcript PDF

2.2 Input Handling & Constraints (§3.1 compliance)

- * Only image/jpeg and image/png are accepted; any other MIME type raises an `UnsupportedTypeError` surfaced as a user-friendly toast.
- * Images whose longest side exceeds 1024 px are gracefully resized (aspect-ratio preserving) rather than rejected, and the user is informed of the applied resize in both a toast and the caption message.
- * Pure three-channel RGB is assumed; no multi-spectral/SAR handling is needed.

3. Vision Pipeline (Model Choices)

The problem statement forbids paid commercial APIs and permits open-source, locally-hosted inference. Because the target deployment is a zero-backend static

site, we implement a deterministic, explainable classical-CV pipeline rather than shipping multi-hundred-MB neural weights to the browser. This choice maximises reproducibility, latency (sub-100 ms per image), and auditability — every answer can be traced to measurable pixel evidence. The architecture is deliberately modular so that a WebGPU/ONNX model (e.g. a quantised YOLO or Grounding DINO) can be dropped into engine.ts behind the same AnalysisResult interface without touching the UI or VQA layers.

3.1 Segmentation

1. The prepared image is drawn to an offscreen canvas and divided into an adaptive grid (12–48 cells per axis, "H22 px cells").
2. Each cell's mean RGB and an intra-cell luminance variance (a texture proxy) are computed by sub-sampling every 2nd pixel.
3. Cells are classified in HSV space into one of the six target classes, plus auxiliary sky/unknown labels that are excluded from the target set:

Class	Heuristic (HSV + texture)
vegetation	green hue 65–165°, S "e 0.18
water body	blue/teal hue 175–260°, S "e 0.15, low variance
open ground	warm hue 20–55°, mid saturation, mid-high value
building	high brightness and high texture variance, or saturated red rooftops
road	low saturation (grey), mid brightness
vehicle	see §3.3

3.2 Bounding Boxes (§2.2 compliance)

Region-like classes (water, vegetation, building, road, open ground) are grouped into axis-aligned bounding boxes via 4-connected component labelling over the class grid (BFS flood fill). Components below a per-class minimum-cell threshold are discarded to suppress noise. Each retained box carries:

- * a confidence derived from the mean per-cell match strength,
- * a key attribute (size bucket + dominant colour name), and
- * an area fraction of the whole image.

Boxes are rendered as coloured overlays *inside the chat/image view* (never a separate download), with class label and confidence badges, satisfying §5.

3.3 Vehicle Detection

Vehicles are small, high-contrast objects. We flag cells with high local variance that are either strongly saturated, very dark, or very bright, and adjacent to a driveable surface (road / open ground / building). Candidates are ranked by confidence and capped to suppress texture noise.

3.4 Caption Generation (§2.1 compliance)

Captions are synthesised from the coverage statistics and object counts. The dominant class leads the sentence, followed by up to three secondary classes, plus a spatial/scene hint (e.g. "the layout suggests a built-up urban area"). Captions are factual, specific, and non-technical, and are shown immediately after upload, before any user query.

4. VQA Engine (§2.3 compliance)

A single unified chat entry point handles all query types — the user never switches modes. answerQuery() infers intent from the query text and returns a grounded answer plus an optional set of classes to highlight on the overlay.

Intent	Trigger examples	Grounding
Caption / describe	"describe the image"	precomputed caption

Detection ("mark all")	"mark all vehicles", "highlight water"	filter + overlay
Binary (yes/no)	"is there a road?"	count / coverage threshold
Numeric	"how many buildings?"	detection counts
Coverage	"what fraction is vegetation?"	per-class pixel coverage
Attribute	"what colour is the largest building?"	dominant-colour of largest box
Out-of-scope	"how many people?"	graceful class-scope message

Class synonym resolution maps natural vocabulary (car/truck/bus !' vehicle, lake/river/pond !' water body, etc.) onto the six canonical classes.

Multi-turn context (bonus). The last-referenced class is retained between turns, so a follow-up such as *"how many of those are visible?"* resolves against the previously discussed class.

Object-level queries (bonus). Clicking any bounding box (on the image or in the detection list) emits a follow-up chat message describing that specific object's class, confidence, colour, and size.

5. Interface Requirements Compliance (§5)

Requirement	Status
Image upload (button + drag-and-drop)	' UploadZone
Automatic caption before any query	' posted on upload
Bounding-box overlay rendered on image in-window	' ImageViewer
Per-object description (class, confidence, attribute)	' detection panel + c
QnA responses in chat bubbles	' chat column
Session history visible	' scrollable transcript
Error handling (bad type, oversize, out-of-scope)	' toasts + scoped replie
Bonus: multi-turn context	'
Bonus: object-level click queries	'
Bonus: exportable PDF report	' exportReport.ts (annotated image + transco

6. Results & Discussion

The classical pipeline runs in well under 100 ms for a 1024×1024 image on a commodity laptop, entirely on the main thread. It is strongest on scenes with clear colour separation (vegetation, water, open ground) and provides honest, reproducible coverage estimates. It degrades gracefully on ambiguous textures (e.g. shadowed buildings vs. road), where confidence scores communicate uncertainty rather than over-claiming. Because the pipeline is deterministic, demo behaviour during the live session is fully predictable.

Limitations. Heuristic segmentation cannot match a fine-tuned neural detector on cluttered urban scenes or heavily occluded vehicles. This is an accepted trade-off given the zero-backend, no-paid-API deployment target and the problem statement's allowance for graceful degradation.

Future work. The AnalysisResult contract isolates the model layer, so a quantised open-source detector (YOLOv8-nano / Grounding DINO) can be run in the browser via ONNX Runtime Web or transformers.js with no UI changes; fine-tuning on public overhead datasets (DOTA, iSAID) would target the innovation component.

7. Credits & Compliance

* No paid commercial vision API is used in the inference path (§3.2 compliant).

* Open-source libraries: React, Tailwind CSS, Radix UI / shadcn-ui components, lucide-react icons, jsPDF (report export). All under permissive licenses.

* Algorithms (HSV colour segmentation, connected-component labelling) are standard classical computer-vision techniques.

8. Running & Deploying

The app is a static Vite React project.

- * Development: the dev server is managed by the hosting environment.
- * Production build: pnpm build emits static assets to dist/, deployable to any static host (Netlify, Vercel, GitHub Pages, S3+CloudFront). No server, database, or API keys are required.
- * Privacy: all analysis is in-browser; images never leave the device.